Reading the Agents dashboard

Every panel explained with everyday examples, and what a senior engineer looks for

http://127.0.0.1:8000/#agents · unlock with the admin token from backend/.env · every number in this deck is a real screenshot

The one idea: CCTV and order tickets for a kitchen

Think of Crickbot as a restaurant kitchen. Every fan question is an order. Observability means you can always answer: what happened to that order, how long did it take, and where did it go wrong?

Logs

The kitchen diary

One line for every event, written as it happens: "order 42 sent to the grill", "grill out of gas".

Where: Logs panel · backend/var/app.log

Metrics

The scoreboard on the wall

Counts and averages: orders per hour, % sent back, average wait.

Where: KPI cards · charts · /metrics · CSVs

Traces

A stopwatch on every order

Who touched this order, in what order, and for how long.

Where: Drill-down timeline · trace waterfall

Meet the kitchen: seven agents, one job each

Crickbot · *Head waiter*

Takes the fan's question and delivers the answer. One run per question: its time is the fan's whole wait.

Planner · *Prep cook*

Checks the pantry first: database lookups and record-book search, about 0.1 s.

Router · *Expediter*

Reads the ticket and decides which stations it needs: live, stats or rules. Instant.

Live · *Runner at the TV*

Reads the live match: score, last balls, win chance (weather and pitch are simulated).

Stats analyst · *Archivist*

15 record-book tools: player stats, death overs, match-ups, venues, toss...

Rules expert · *Referee*

Looks up the rule book: no ball, LBW, DRS...

Commentator · *Head chef*

Writes the final answer from everything gathered. The only one using the AI model: the slow part.

Fast questions skip most of the kitchen.
"What's the score?" or "Who won IPL 2016?" are answered by Crickbot + Planner alone in about 0.2 s. Only questions that need explaining reach the Commentator.

One real question, end to end

"How does Kohli usually get out?" (asked while I had switched the stats database off on purpose)

Crickbot
3.66 s
the fan's total wait

→

Planner
82 ms
pantry check

→

Router
0 ms
"needs stats"

→

Stats analyst
1 ms
tool failed: database off

→

Commentator
3.47 s
wrote the answer



Handoffs: each agent passes work to the next (Crickbot → Planner → Router → Stats analyst → Commentator).

Where the time went: 95% in the Commentator, the AI model writing. Everything else took under 0.1 s.

The failure didn't stop the answer: the tool error is recorded, the database answer is skipped, and the fan still got a reply.

One ID ties it together: correlation c1fcfbba8bb8fee6 finds every agent, trace and log line of this question.

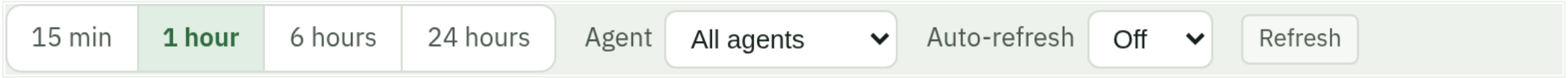
Words you'll see on the dashboard

Word	What it means	In the kitchen
Run	One agent doing its job once	*One cook doing one task*
Turn	One fan question = one Crickbot run	*One customer's order*
Run id	Unique id of one run	*The task's slip number*
Correlation id	Shared by every run of one question	*The order-ticket number*
Trace id	The same question in the tracing system	*The order's stopwatch card*
Handoff	One agent passing work to the next	*"Order to the grill!"*
Latency	How long something took	*How long the customer waited*
p50 / p95	Half / 95 of 100 were faster than this	*Typical wait / worst normal wait*
Token	A piece of a word the AI reads or writes	*Ingredients in, words on the plate out*
Tool call	An agent looking something up	*Fetching from the pantry*
Retry	Calling the AI again after it failed	*Re-firing a burnt dish*
Status	ok · error · degraded · blocked · cancelled	*Served · failed · simpler dish · refused · walked out*

Map of the page: read it top to bottom

1. **Toolbar** time range · agent filter · auto-refresh
2. **KPI cards** 8 headline numbers
3. **Charts** runs · latency · tokens · tool calls
4. **Agent status** health of each agent
5. **Handoff graph** who passes work to whom
6. **Errors** grouped failures
7. **Tools** every lookup's stats
8. **Executions** every run, searchable → click for the drill-down
9. **Logs** the diary, live
10. **CSV exports** files for Excel

1 · Toolbar: what you're looking at



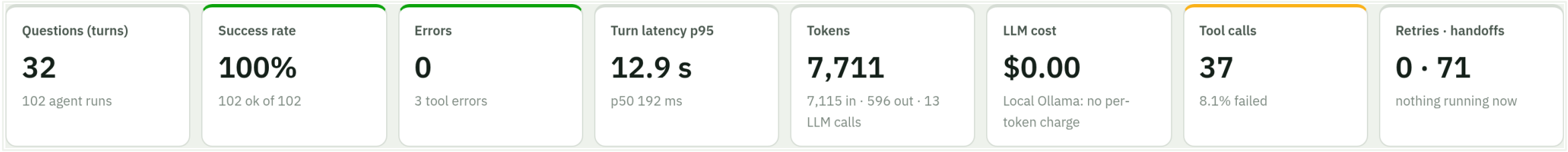
1. **Time range: 15 min · 1 hour · 6 hours · 24 hours**
   How far back every card, chart and table looks. Like rewinding the CCTV. Start with 1 hour; switch to 15 min while you test something.

2. **Agent filter**
   Narrows the whole page to one agent, e.g. only the Commentator. Same as clicking an agent's status card or a node in the handoff graph.

3. **Auto-refresh: Off · 5 s · 15 s · 30 s**
   How often the page fetches new data. It pauses while the browser tab is hidden. Turn it Off to read a table without it moving.

4. **"Updated 15:38 · 102 runs in memory (… reloaded from CSV)"**
   When the data last arrived. After a restart the last 24 hours are reloaded from the CSV files, so history survives.

2 · KPI cards: the scoreboard (1 of 2)



Questions (turns) · 32

How many fan questions were answered. Below it: 102 agent runs, because one question uses several agents.

Orders served today, and how many kitchen tasks they took.

Success rate · 100%

Share of agent runs that finished OK. Card edge: green ≥ 99%, amber 90-99%, red below 90%.

Dishes that left the kitchen without being sent back.

Errors · 0 (3 tool errors)

Agent runs that failed outright. A failed lookup inside a run counts as a tool error, not an agent error, when the agent still finished.

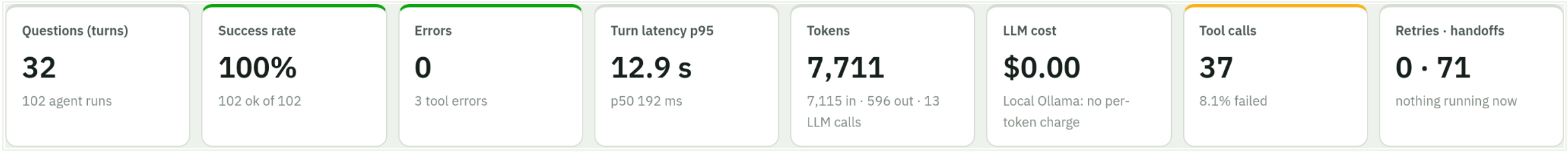
The grill ran out of gas (tool error) but the chef still served a salad (agent OK).

Turn latency p95 · 12.9 s (p50 192 ms)

Wait for the whole question. p50: half waited less. p95: 95 of 100 waited less. A big gap = most answers are instant, a few use the AI.

Most coffees are instant; the occasional cooked meal takes 13 s.

2 · KPI cards: the scoreboard (2 of 2)



Tokens · 7,711 (7,115 in · 596 out)

How much the AI read (in) and wrote (out) over 13 LLM calls. In is always far bigger: the agents hand it the facts, it writes a short answer.

Ingredients carried to the chef vs words written on the menu card.

LLM cost · $0.00

Tokens × price. The AI runs locally on this laptop (Ollama), so there is no per-token bill. Set a price in backend/.env to estimate a hosted model.

Cooking with vegetables from your own garden: no supplier invoice.

Tool calls · 37 (8.1% failed)

Every lookup the agents made. The 3 failures were the database I switched off on purpose. Amber edge = some failed.

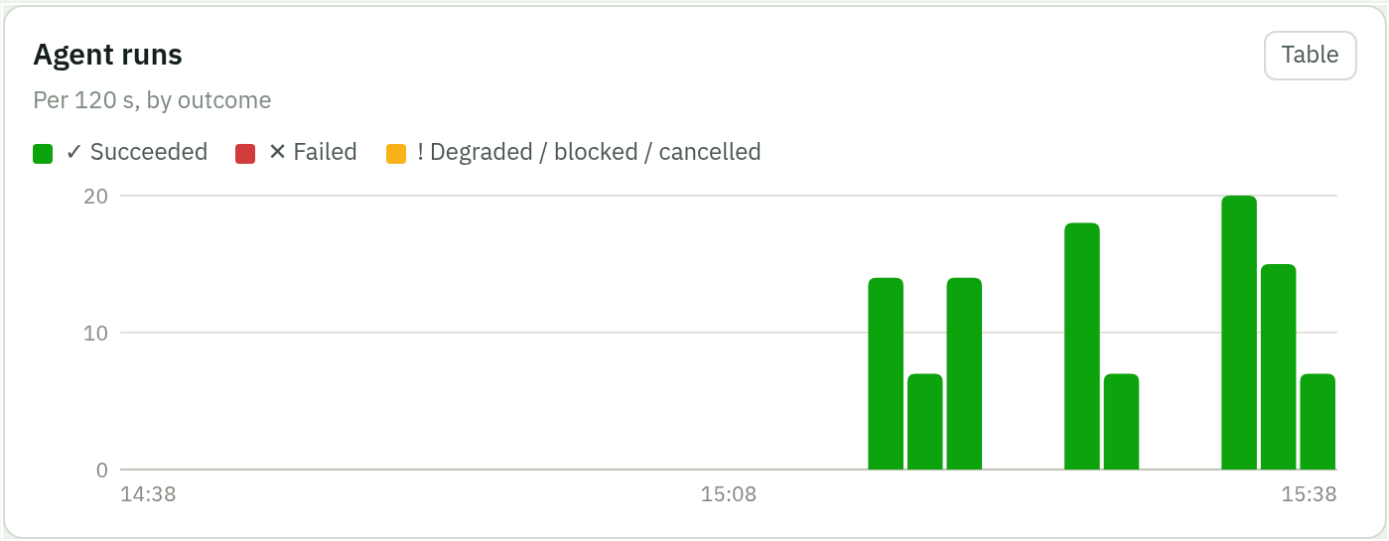
Trips to the pantry, and how many came back empty.

Retries · handoffs · 0 · 71

Retries: the AI was called again after a failure (0 is good). Handoffs: work passed between agents (normal, grows with traffic). Bottom line: anything running right now.

Re-fired dishes · tickets passed between stations.

3 · Agent runs chart: busy, and how did it go?



1 **Each bar = one time slot**

With "1 hour" selected a slot is 2 minutes. Taller bar = more agent runs in that slot.

2 **Colours = outcome**

Green ✓ succeeded · red ✕ failed · amber ! degraded, blocked or cancelled.

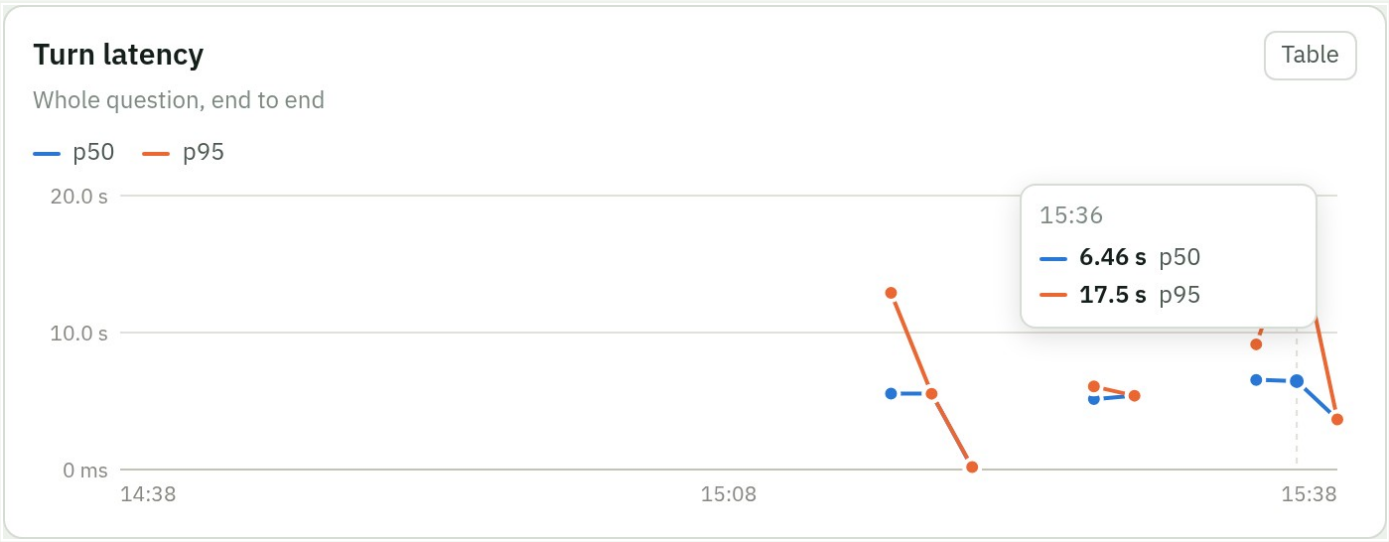
3 **Hover or use ← → keys**

Shows the exact counts for that slot. "Table" shows every number.

Everyday example *The kitchen's tally board: one column per 2 minutes, green for dishes served, red for dishes sent back.*

What a senior looks for Red appearing where there was none · amber growing (the AI is down or busy and fans get simpler answers) · a sudden drop to nothing while fans are still asking (something upstream is broken).

4 · Turn latency: how long fans wait



1. **Blue = p50, the typical wait**
   In the 15:36 slot: 6.46 s. Half the questions in that slot were answered faster.

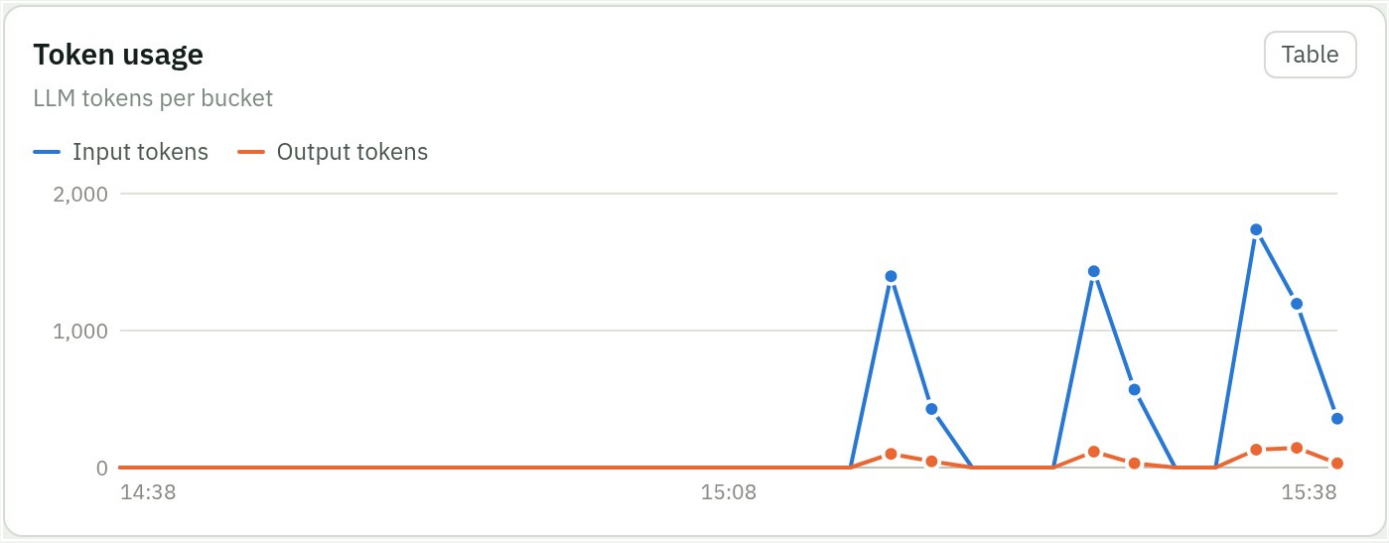
2. **Orange = p95, the slow wait**
   17.5 s: 95 of 100 were faster. This is the number fans complain about.

3. **Gaps are normal**
   A slot with no questions has no latency, so the line breaks. Dips near 0 are instant answers.

Everyday example *Line up 100 customers by how long they waited. The 50th person's wait is p50; the 95th person's is p95.*

What a senior looks for p95 climbing while traffic is flat (the AI is getting slower) · p50 jumping up (even simple questions are slow, so look at the Planner or the database) · use the Agent filter to see which agent's latency moved.

5 · Tokens and cost: what the AI reads and writes



1. **Blue = input tokens**

   What we hand the AI: instructions plus the facts the agents found. Usually hundreds per question.

2. **Orange = output tokens**

   What the AI writes: the answer. Short by design (2-4 sentences).

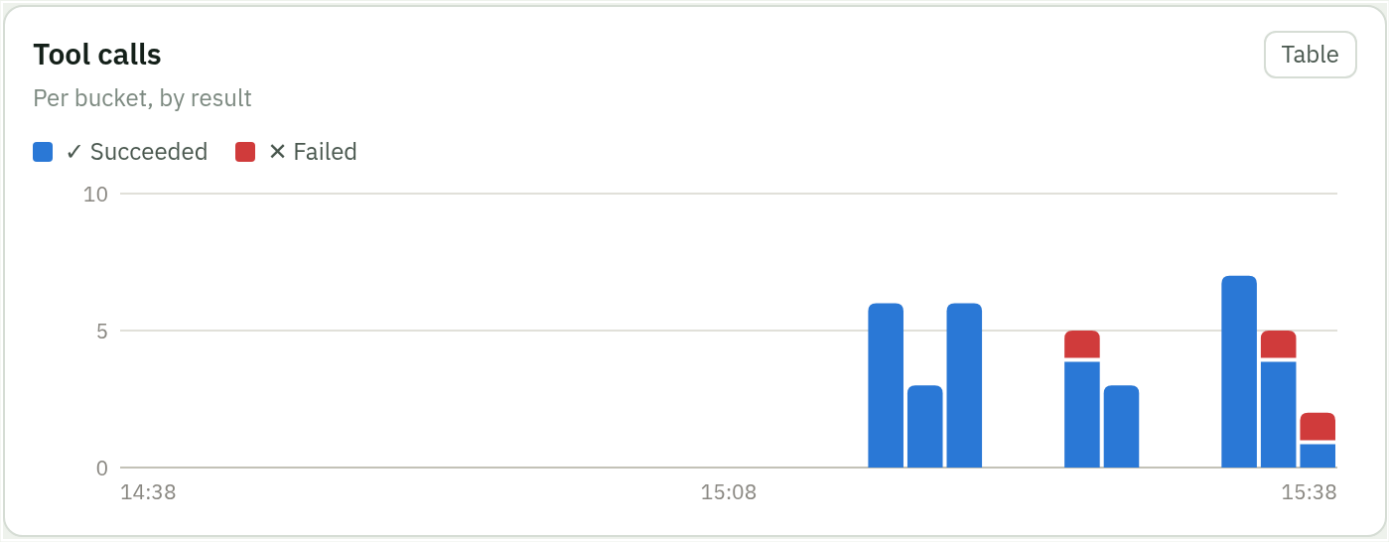
3. **Spikes = AI questions**

   Instant answers use no tokens at all, so the line sits at 0 between them.

Everyday example *A token is roughly ¾ of a word. Input is the ingredients carried to the chef; output is the words written on the plate.*

Why cost is $0.00 The model runs on this laptop, so tokens cost electricity, not money. With a hosted model, cost = tokens × price per million: set CRICKET_LLM_PRICE_IN_PER_MTOK and _OUT_PER_MTOK in backend/.env and the cost card and CSVs fill in.

6 · Tool calls: the agents' lookups



1. **Blue ✓ = lookups that worked**
   Database queries, record-book searches, live-score reads.

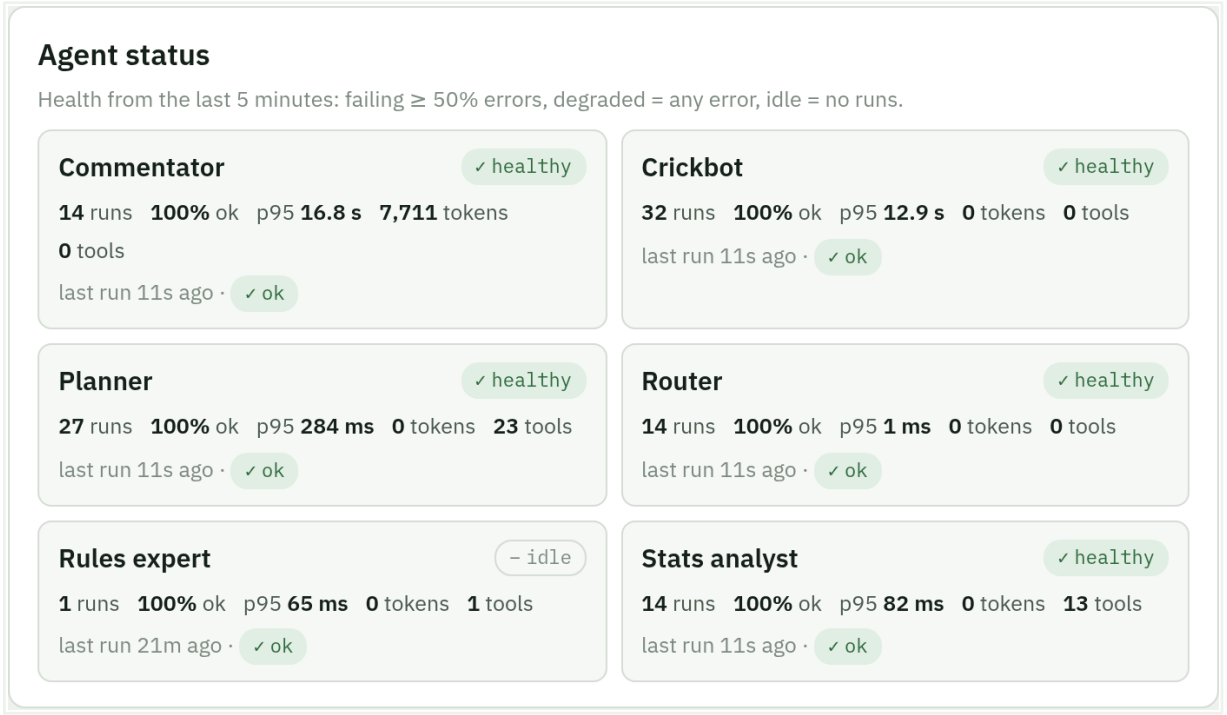
2. **Red ✕ = lookups that failed**
   Here: the stats database I switched off on purpose ("ChaosError").

3. **Most lookups take milliseconds**
   The Tools table below shows each tool's average and p95 time.

Everyday example *Trips to the pantry. Blue: came back with the ingredient. Red: the shelf was empty or the door was locked.*

What a senior looks for Red tied to one tool (check the Tools and Errors tables) · red across all tools (a shared dependency such as the database is down; the Dashboard tab shows component health) · blue dropping to zero while questions continue.

7 · Agent status: a health check per agent



✓ healthy — Ran in the last 5 min with no errors

! degraded — At least one error in the last 5 min

× failing — Half or more of its runs failed in the last 5 min

– idle — No runs in the last 5 min (normal when nobody asks)

Each card: runs · % ok · p95 · tokens · tools · last run and its status.

Click a card to filter the whole page to that agent; click again to clear.

Everyday example *The kitchen display above each station: green light while it works, amber after a mistake, red when it keeps failing, off when idle.*

8 · Handoff graph: how work flows between agents



1. **Boxes = agents**

   Under each name: health icon and number of runs. Dashed box = no runs in this range.

2. **Lines = handoffs**

   Work passed from left to right. Thicker line = that path is used more. The number is the count.

3. **Read it left to right**

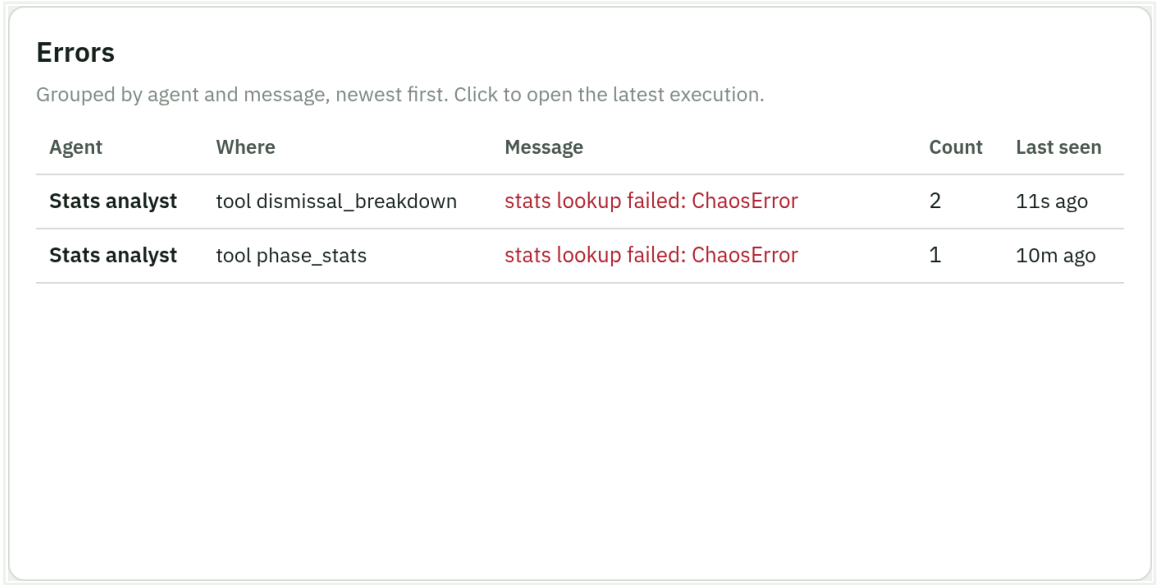
   Crickbot → Planner → Router → desks (Live, Stats, Rules) → Commentator.

4. **Click a box**

   Filters the dashboard to that agent.

Everyday example *The order rail in a kitchen: tickets slide from the waiter to the prep cook to the stations to the chef. Busy rails get worn.*

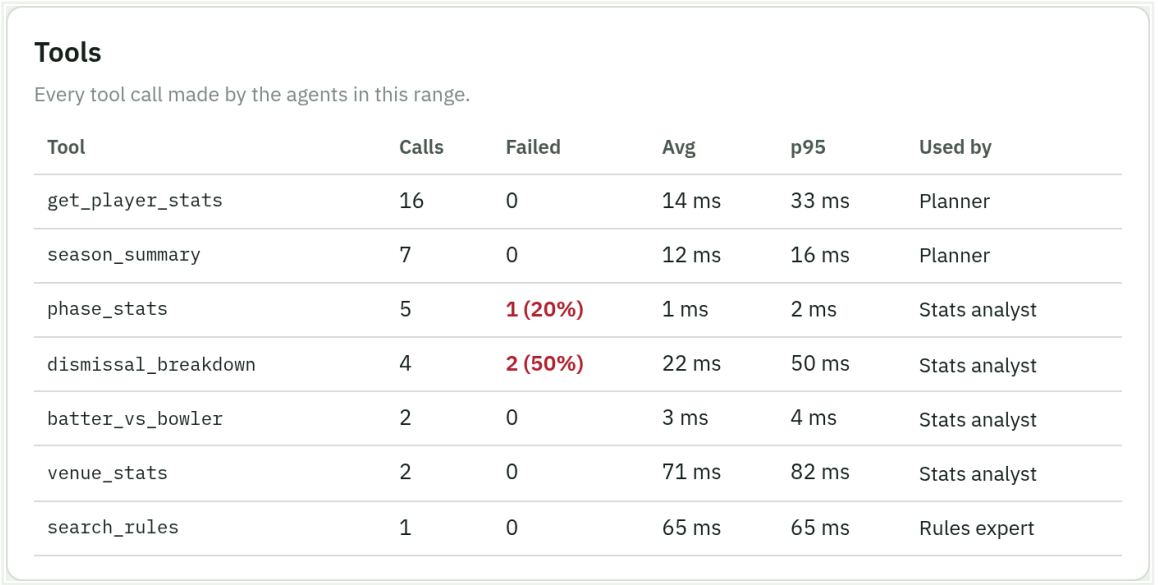
9 · Errors and Tools: what failed, and where



Errors

Grouped by agent and message, newest first. Click to open the latest execution.

Agent	Where	Message	Count	Last seen
Stats analyst	tool dismissal_breakdown	stats lookup failed: ChaosError	2	11s ago
Stats analyst	tool phase_stats	stats lookup failed: ChaosError	1	10m ago



Tools

Every tool call made by the agents in this range.

Tool	Calls	Failed	Avg	p95	Used by
get_player_stats	16	0	14 ms	33 ms	Planner
season_summary	7	0	12 ms	16 ms	Planner
phase_stats	5	1 (20%)	1 ms	2 ms	Stats analyst
dismissal_breakdown	4	2 (50%)	22 ms	50 ms	Stats analyst
batter_vs_bowler	2	0	3 ms	4 ms	Stats analyst
venue_stats	2	0	71 ms	82 ms	Stats analyst
search_rules	1	0	65 ms	65 ms	Rules expert

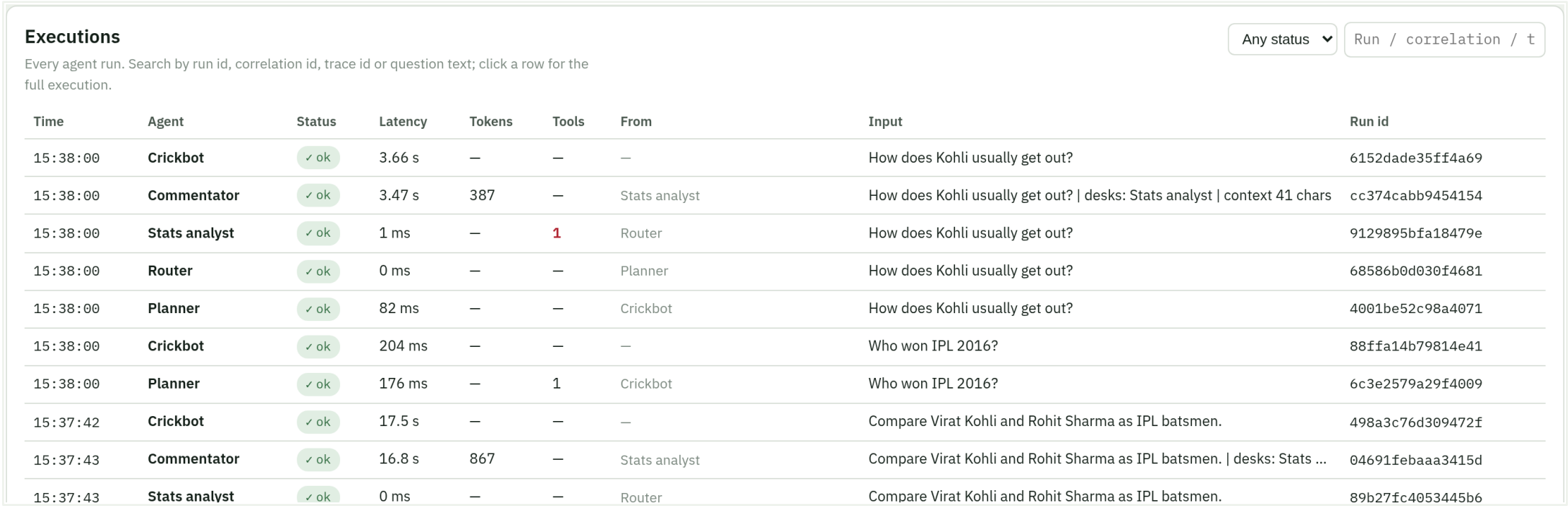
Errors

- Grouped: the same failure on the same agent is one row with a count.
- "Where" says run or tool: "tool dismissal_breakdown" = a lookup failed.
- Click a row to open the latest execution with that error.
- "ChaosError" = I pulled the plug on purpose to test this panel.

Tools

- Calls, failed (with %), average and p95 time per tool.
- "Used by" shows which agent calls it.
- Slow p95 = a heavy query; failed % = a broken dependency.
- Mock tools (weather, pitch) also appear here when used.

10 · Executions: every run, searchable



Executions

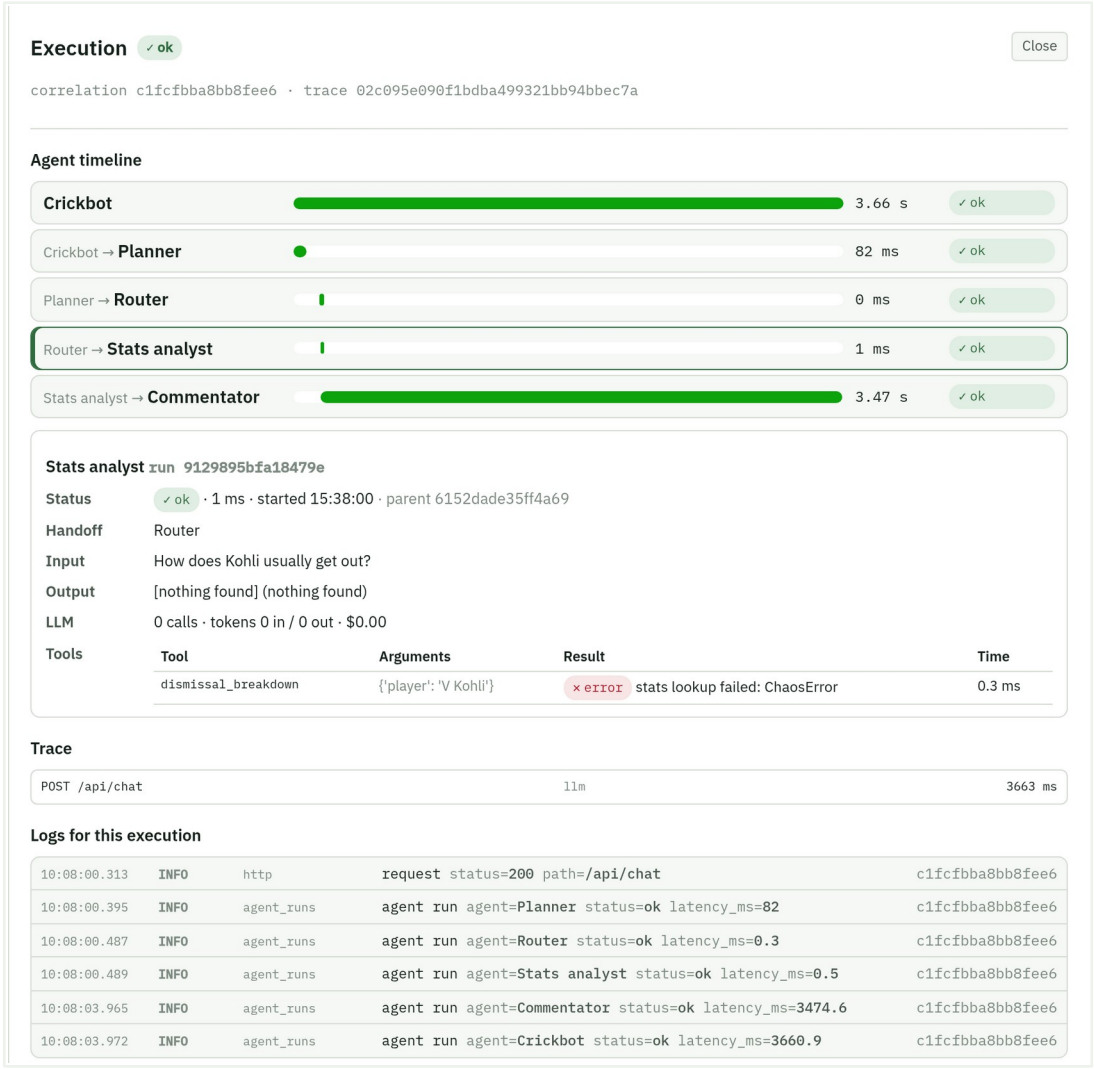
Every agent run. Search by run id, correlation id, trace id or question text; click a row for the full execution.

Any status | Run / correlation / t

Time	Agent	Status	Latency	Tokens	Tools	From	Input	Run id
15:38:00	**Crickbot**	✓ ok	3.66 s	—	—	—	How does Kohli usually get out?	6152dade35ff4a69
15:38:00	**Commentator**	✓ ok	3.47 s	387	—	Stats analyst	How does Kohli usually get out? | desks: Stats analyst | context 41 chars	cc374cabb9454154
15:38:00	**Stats analyst**	✓ ok	1 ms	—	1	Router	How does Kohli usually get out?	9129895bfa18479e
15:38:00	**Router**	✓ ok	0 ms	—	—	Planner	How does Kohli usually get out?	68586b0d030f4681
15:38:00	**Planner**	✓ ok	82 ms	—	—	Crickbot	How does Kohli usually get out?	4001be52c98a4071
15:38:00	**Crickbot**	✓ ok	204 ms	—	—	—	Who won IPL 2016?	88ffa14b79814e41
15:38:00	**Planner**	✓ ok	176 ms	—	1	Crickbot	Who won IPL 2016?	6c3e2579a29f4009
15:37:42	**Crickbot**	✓ ok	17.5 s	—	—	—	Compare Virat Kohli and Rohit Sharma as IPL batsmen.	498a3c76d309472f
15:37:43	**Commentator**	✓ ok	16.8 s	867	—	Stats analyst	Compare Virat Kohli and Rohit Sharma as IPL batsmen. | desks: Stats ...	04691febaaa3415d
15:37:43	**Stats analyst**	✓ ok	0 ms	—	—	Router	Compare Virat Kohli and Rohit Sharma as IPL batsmen.	89b27fc4053445b6

1. **Search box**
   Paste a run id, correlation id or trace id, or type words from the question ("malinga").
2. **Status filter**
   Show only errors, only degraded, and so on.
3. **Click any row**
   Opens the drill-down for that whole question, with the clicked agent selected.

11 · Drill-down: the full story of one question



Execution ✓ ok — Close

correlation c1fcfbba8bb8fee6 · trace 02c095e090f1bdba499321bb94bbec7a

Agent timeline

Crickbot	3.66 s	✓ ok
Crickbot → Planner	82 ms	✓ ok
Planner → Router	0 ms	✓ ok
Router → Stats analyst	1 ms	✓ ok
Stats analyst → Commentator	3.47 s	✓ ok

Stats analyst run 9129895bfa18479e

Status ✓ ok · 1 ms · started 15:38:00 · parent 6152dade35ff4a69
Handoff Router
Input How does Kohli usually get out?
Output [nothing found] (nothing found)
LLM 0 calls · tokens 0 in / 0 out · $0.00

Tools

Tool	Arguments	Result	Time
dismissal_breakdown	{'player': 'V Kohli'}	× error stats lookup failed: ChaosError	0.3 ms

Trace

POST /api/chat	llm	3663 ms

Logs for this execution

10:08:00.313	INFO	http	request status=200 path=/api/chat	c1fcfbba8bb8fee6
10:08:00.395	INFO	agent_runs	agent run agent=Planner status=ok latency_ms=82	c1fcfbba8bb8fee6
10:08:00.487	INFO	agent_runs	agent run agent=Router status=ok latency_ms=0.3	c1fcfbba8bb8fee6
10:08:00.489	INFO	agent_runs	agent run agent=Stats analyst status=ok latency_ms=0.5	c1fcfbba8bb8fee6
10:08:03.965	INFO	agent_runs	agent run agent=Commentator status=ok latency_ms=3474.6	c1fcfbba8bb8fee6
10:08:03.972	INFO	agent_runs	agent run agent=Crickbot status=ok latency_ms=3660.9	c1fcfbba8bb8fee6

1. **Header**
   Overall status, correlation id and trace id of the question.

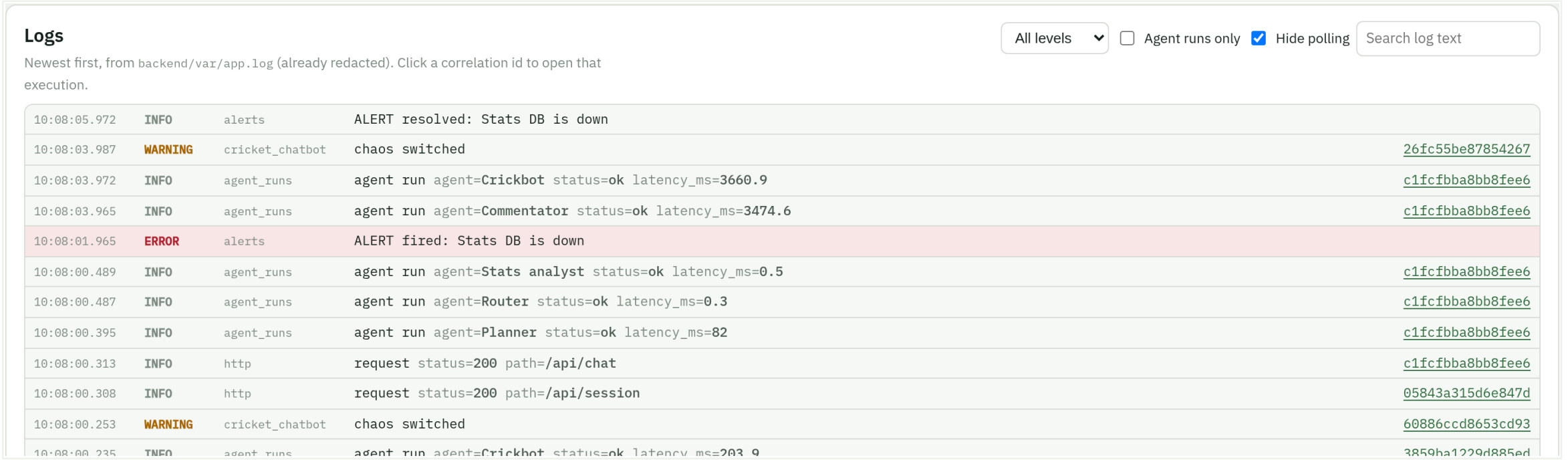
2. **Agent timeline**
   Each agent as a bar on one clock: where the time went. Who handed over is shown before the name.

3. **Selected run**
   Input, output, LLM calls, tokens, cost, retries, and every tool call with its arguments, result and time.

4. **Trace**
   The same question in the tracing system (kept for the latest 50 questions).

5. **Logs for this execution**
   Every diary line with this correlation id, in order.

12 · Logs: the kitchen diary, live



Logs

Newest first, from backend/var/app.log (already redacted). Click a correlation id to open that execution.

All levels | Agent runs only | Hide polling | Search log text

10:08:05.972	INFO	alerts	ALERT resolved: Stats DB is down	
10:08:03.987	WARNING	cricket_chatbot	chaos switched	26fc55be87854267
10:08:03.972	INFO	agent_runs	agent run agent=Crickbot status=ok latency_ms=3660.9	c1fcfbba8bb8fee6
10:08:03.965	INFO	agent_runs	agent run agent=Commentator status=ok latency_ms=3474.6	c1fcfbba8bb8fee6
10:08:01.965	ERROR	alerts	ALERT fired: Stats DB is down	
10:08:00.489	INFO	agent_runs	agent run agent=Stats analyst status=ok latency_ms=0.5	c1fcfbba8bb8fee6
10:08:00.487	INFO	agent_runs	agent run agent=Router status=ok latency_ms=0.3	c1fcfbba8bb8fee6
10:08:00.395	INFO	agent_runs	agent run agent=Planner status=ok latency_ms=82	c1fcfbba8bb8fee6
10:08:00.313	INFO	http	request status=200 path=/api/chat	c1fcfbba8bb8fee6
10:08:00.308	INFO	http	request status=200 path=/api/session	05843a315d6e847d
10:08:00.253	WARNING	cricket_chatbot	chaos switched	60886ccd8653cd93
10:08:00.235	INFO	agent_runs	agent run agent=Crickbot status=ok latency_ms=203.9	3859ba1229d885ed

A real story in this screenshot (read bottom to top)

10:08:00 chaos switched: I turned the stats database off from the Dashboard tab

10:08:00 Planner, Router, Stats analyst ran: the lookup failed, the agents carried on

10:08:01 ERROR ALERT fired: Stats DB is down (the monitor noticed within ~2 s)

10:08:03 Commentator and Crickbot finished: the fan got an answer

10:08:05 ALERT resolved after I switched it back on

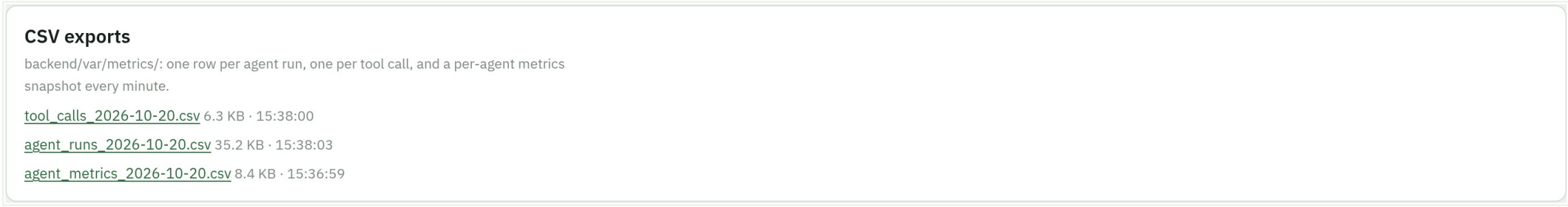
Levels: INFO normal · WARNING odd · ERROR broken (red row)

Hide polling hides the dashboard's own requests

Agent runs only shows one line per agent run

Click an id to open that execution

13 · CSV exports: the same data in Excel



`agent_runs_<date>.csv`

One row per agent run

run id, correlation id, trace id, agent, status, latency, tokens, cost, tool calls, retries, handoff, input/output summary, error

"*Which agent was slowest yesterday?*"

`tool_calls_<date>.csv`

One row per tool call

time, run and correlation ids, agent, tool, ok, ms, arguments, error

"*How often did phase_stats fail this week?*"

`agent_metrics_<date>.csv`

Agent totals each minute

runs, ok, errors, success rate, p50/p95/max latency, tokens, LLM calls, tool calls, retries

"*Draw success rate per hour in Excel*"

When something goes wrong: where to look

Symptom	First look	Then
Fans say answers are slow	Latency chart → is p95 or p50 up?	Open a slow execution → the longest bar is the culprit (usually Commentator = AI model)
Red appears on the Agent runs chart	Errors table → which agent and message?	Click the row → drill-down → error text and logs
An answer was wrong	Executions → search the fan's words	Drill-down → what did the desks find? Did a tool fail?
Tool failures rising	Tools table → one tool or all?	All tools = shared dependency down → Dashboard tab components
Cost or tokens jump	Token chart → input or output?	Input up = prompts too big · output up = answers too long

Rule of thumb: start from the number, click down to one execution, read its timeline, then its logs. Never guess from the average.

Questions people ask, and short answers

Is any of this fake data?

No. Every number comes from real agent runs, logs and traces. Weather and pitch tools are simulated and labelled MOCK.

Success is 100% but tools failed. Why?

Agents that finish OK are successes even if a lookup failed inside them; those are counted as tool errors.

Why is the Router 0 ms?

It decides with fixed rules first and only asks the AI when no rule matches.

Why is the trace missing in a drill-down?

Only the latest 50 traces are kept in memory. Runs and logs are still there.

What happens after a restart?

The last 24 h are reloaded from the CSV files. CSVs are kept for 30 days.

Can it leak secrets?

Passwords, API keys, tokens, emails and phone numbers are redacted before anything is written.

The 30-second daily check

1. **Success rate green?**
   ≥ 99% and no red bars
2. **p95 latency steady?**
   no climb at flat traffic
3. **Errors table empty?**
   if not, open the newest
4. **All agents healthy or idle?**
   no amber or red cards
5. **Tool failures near zero?**
   and none on one tool